

PokeArb Japan: Full Application Architecture and Vision System Report

Local project report

May 12, 2026

Abstract

This report documents the current PokeArb Japan application from the user interface down through the Next.js API layer, pricing integrations, OCR, local card identification, embedding retrieval, reranking, and operational setup. It is written from the inspected code in the V2 worktree and focuses on how the app actually works today, not only on the intended roadmap. The most important design point is that the app is not just a price calculator and it is not just an image classifier. It is a local deal-assistance system: a mobile-first frontend captures shop photos, backend routes normalize the request, OCR extracts yen prices, a local retrieval engine identifies Pokemon cards from real photos, pricing sources estimate raw and graded reference values, and the calculator turns those values into a buy/maybe/skip decision.

Contents

1	Executive Summary	2
2	Repository and Runtime Context	2
3	Application Purpose	3
4	High-Level System Architecture	4
5	Frontend Architecture	4
5.1	Main File	4
5.2	Screens	5
5.3	Local State	5
5.4	Camera and Scanner UX	5
5.5	Theme System	6
6	Frontend Styling and Liquid Theme	6
7	Deal Calculation Model	6
7.1	Default Settings	6
7.2	Margin Formula	7
7.3	Routes	7
7.4	Warnings	7
8	Next.js Backend API Layer	8
9	Manual Card Search	8

10 Reference Pricing System	9
10.1 Main Merge Function	9
10.2 Currency Conversion	9
10.3 Source Priority	9
10.4 Cardmarket Live Pricing	10
10.5 TCGdex Pricing	10
10.6 PokemonTCG API Pricing	10
10.7 PokePrices	10
10.8 PriceCharting	10
11 Price OCR Scanner	11
11.1 Frontend Flow	11
11.2 Next.js Price OCR Route	11
11.3 GPU Price OCR Sidecar	11
11.4 White Price Tag Detection	12
11.5 Yen Candidate Extraction	12
12 Card Image Recognition System	12
12.1 Why Retrieval Instead of a Giant Classifier	12
12.2 Data Sources	13
12.3 Card Metadata	14
12.4 Current Japanese Coverage Caveat	14
13 Image Preprocessing and Geometry	15
13.1 Target Size	15
13.2 Candidate Crops	15
13.3 Contour Detection	15
13.4 Why No Trained Detector Yet	16
14 Classical Visual Features	16
14.1 Perceptual Hashes	16
14.2 Color Histogram	16
14.3 ORB Keypoint Matching	16
15 DINOv2 Embeddings	17
15.1 What the Embedding Model Does	17
15.2 Why DINOv2 Helps	17
15.3 Reference Embedding Variants	17
15.4 Query Embedding Variants	18
15.5 Current Search Implementation	18
15.6 RTX 5090 Role	18
16 Matcher Scoring and Reranking	18
16.1 Coarse Score	18
16.2 Rerank Stage	19
16.3 V2 Reranker	19
16.4 Unimplemented or Placeholder Evidence	20
17 Card OCR and ROI Extraction	20
17.1 Purpose	20
17.2 Current Implementation	20
17.3 Current Runtime Limitation	20

18 Recognition Output Contract	21
19 Next.js Integration for Card Scanning	22
19.1 Route	22
19.2 Provider Order	22
19.3 Local Matcher Execution	22
19.4 Parsing and Mapping	22
20 Scan Result Merging	23
21 Marketplace Query Strategy	23
21.1 North America	23
21.2 Europe	23
22 History and Persistence	23
23 Testing Coverage	24
24 Current Verification State	24
25 Deep Learning Inventory	24
26 What Has Not Been Trained From Scratch	25
27 Where Training May Become Useful	25
28 Failure Modes	26
28.1 Recognition Failure Modes	26
28.2 Price OCR Failure Modes	26
28.3 Pricing Failure Modes	27
29 Recommended Evaluation Harness	27
29.1 Dataset	27
29.2 Labels	27
29.3 Metrics	28
30 Security and Operational Considerations	28
30.1 Local-Only Assumption	28
30.2 Upload Limits	28
30.3 External Credentials	28
30.4 Marketplace Scraping	28
31 Recommended Roadmap	29
31.1 Near-Term	29
31.2 Medium-Term	29
31.3 Long-Term	29
32 Glossary	29
33 Key Files	30
34 Conclusion	30

A	Appendix: Useful Commands	31
A.1	Run App on Port 3000	31
A.2	Build Official Japanese Gallery	31
A.3	Build DINOv2 Embeddings	31
A.4	Match One Photo	31
A.5	Run TypeScript Tests	31
A.6	Run Card Matcher Benchmark	31

1 Executive Summary

PokeArb Japan is a mobile-first web application for checking Pokemon card deals while shopping in Japan. The user can scan or search a card, enter or OCR a yen price, fetch reference market values, and get a route-aware decision for raw flipping, grading to PSA 9, grading to PSA 10, or buying already graded slabs.

The application has three major technical systems:

1. A Next.js/React frontend that manages the app screens, local state, theme system, camera uploads, scan results, deal calculations, and saved history.
2. A backend API layer implemented as Next.js route handlers. These routes call card search, reference pricing, local GPU price OCR, local card recognition, Cardmarket live scraping, and reverse geocoding helpers.
3. A Python vision stack under `tools/card_matcher` and `tools/price_ocr_server`. The card matcher uses OpenCV crop normalization, perceptual hashes, color histograms, ORB feature matching, DINOv2 embeddings, optional PaddleOCR card ROI features, and a weighted reranker. The price OCR server uses PaddleOCR on GPU and specialized price-tag crops.

The strongest part of the current design is that card recognition is a retrieval system instead of a giant end-to-end classifier. A reference gallery is built from official card images. Each reference card receives visual fingerprints and DINOv2 embeddings. A shop photo is cropped and normalized, embedded, compared to the reference gallery, and reranked. This makes the system maintainable: adding more cards is mostly a matter of syncing metadata, downloading reference images, and rebuilding fingerprints/embeddings, rather than retraining a classifier over every print.

The most important current limitation is that full Japanese coverage is now based on the official Japanese card search gallery, but the official gallery currently stores the official card ID as the local identity for many cards. It gives broad visual coverage, but the printed collector number and exact marketplace-ready print identity still need richer parsing from detail pages or another metadata reconciliation layer. Also, the optional card ROI OCR and future set-symbol/variant classifiers are not yet fully integrated into the hosted scan path by default.

2 Repository and Runtime Context

The current implementation described here lives in the V2 worktree:

- Main application: `src/app/page.tsx`
- Global styling and themes: `src/app/globals.css`
- TypeScript business logic: `src/lib`
- Next.js API routes: `src/app/api`
- Card matcher: `tools/card_matcher`
- GPU price OCR sidecar: `tools/price_ocr_server`
- Cardmarket live pricing sidecar: `tools/cardmarket_server`

The package stack is:

Area	Main dependency or tool
Frontend framework	Next.js 16.2.5, React 19.2.4, React DOM 19.2.4
Styling	Tailwind 4 via @tailwindcss/postcss, custom CSS in globals.css
Glass theme	liquid-glass-react 1.1.1
Icons	lucide-react 1.14.0
Browser OCR fallback	tesseract.js 7.0.0
Tests	Vitest 4.0.15
Python card matcher	OpenCV, Pillow, ImageHash, NumPy, Requests, tqdm
GPU embedding extras	PyTorch in the Conda MNE environment, Transformers, Safetensors
GPU OCR	PaddleOCR, configured for PP-OCRv5 server detection and recognition models
Optional vector store	FAISS helper exists, while the MVP still uses a NumPy .npz embedding matrix

The app is intended to be hosted on port 3000 for phone testing. The local matcher is configured through environment variables such as:

```
LOCAL_CARD_MATCHER_DIR=tools/card_matcher
LOCAL_CARD_MATCHER_PYTHON=%USERPROFILE%\anaconda3\envs\MNE\python.exe
LOCAL_CARD_MATCHER_INDEX=tools/card_matcher/cache/ja_official_image_index.json
LOCAL_CARD_MATCHER_EMBEDDING_INDEX=tools/card_matcher/cache/
ja_official_embeddings_facebbook_dinov2-base.npz
LOCAL_CARD_MATCHER_SCRIPT=match_card_v2.py
LOCAL_CARD_MATCHER_TOP=8
LOCAL_CARD_MATCHER_TIMEOUT_MS=300000
LOCAL_CARD_MATCHER_DEVICE=cuda
PRICE_OCR_URL=http://127.0.0.1:8765/ocr-price
```

3 Application Purpose

The app answers a practical in-store question: if a card in a Japanese shop costs a certain yen amount, is it worth buying after currency conversion, selling fees, grading fees, and market uncertainty?

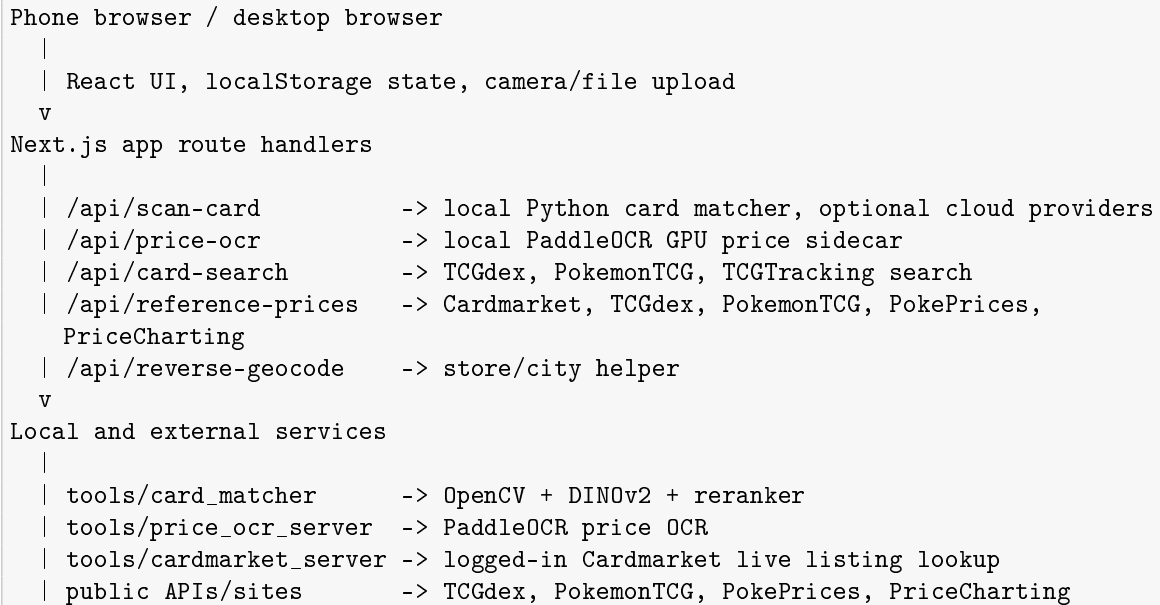
The user workflow is:

1. Open the web app on a phone.
2. Scan a card or manually search/select one.
3. Scan or enter the shop price in yen.
4. Fetch raw and graded reference values.
5. Choose raw or graded in-hand condition.
6. Let the calculator evaluate routes.
7. Save good deals into history.

The app is not trying to be a complete inventory manager yet. It is optimized for fast, repeated decision-making in shops where the user may only have a few seconds per card.

4 High-Level System Architecture

The current architecture can be summarized as:



The browser is intentionally thin for expensive vision work. It can run Tesseract as a fallback, but the serious OCR and image retrieval run server-side so the RTX 5090 and the local cache can be used.

5 Frontend Architecture

5.1 Main File

The main UI is in `src/app/page.tsx`. It is a client component and contains the primary app state, screen rendering, scanner components, settings, history, theme selector, and liquid glass wrappers.

Important frontend functions and components include:

Component or function	Responsibility
<code>App</code>	Root client component. Loads persistent state, manages current screen, theme, liquid background, settings, history, and deal form.
<code>useLiquidMotionState</code>	Tracks scrolling/motion values used by the liquid theme so glass surfaces can react to page movement.
<code>LiquidGlassSurface</code>	Wrapper around <code>liquid-glass-react</code> . Gives repeated app surfaces the same glass behavior while preserving content layout.
<code>AppHeader</code>	Shows app title and lock button.
<code>HomeScreen</code>	Entry screen with conversion summary and shortcuts to deal, PSA, and history.
<code>DealCheckScreen</code>	Core deal calculator and scanner screen.
<code>DealPhotoScanner</code>	Handles camera/file upload, parallel card scan and price OCR, and applies scan results back to the deal form.

<code>HistoryScreen</code>	Displays saved deals from local storage.
<code>SettingsScreen</code>	Exposes formula settings, theme selector, liquid background options, raw condition/language filters.
<code>PsaSlabScreen</code>	PSA slab workflow and cert/card input.

5.2 Screens

The screen type is defined in `src/lib/types.ts`:

```
export type Screen =
  | "home"
  | "deal-check"
  | "psa"
  | "history"
  | "settings";
```

The app is a single page application, not a multi-page route tree. That is appropriate here because the interaction is stateful and repeated: the user moves between Deal, History, and Settings while keeping the same current card/deal context.

5.3 Local State

Persistent state uses `localStorage` through `src/lib/storage.ts`. The helper `useLocalStorageState` is implemented with `useSyncExternalStore`, which means it supports React's external store semantics and also reacts to cross-tab storage events.

Stored app keys include:

- unlocked/lock state
- settings
- theme
- selected liquid background
- history
- store/city context

The frontend stores user-facing state locally because the app is meant to run as a personal tool on the user's machine/phone. There is no database-backed multi-user account model in the current code.

5.4 Camera and Scanner UX

The deal scanner is triggered from `DealPhotoScanner`. When a file/photo is selected, the frontend runs multiple recognition tasks in parallel:

```
Promise.allSettled([
  scanCardImage(file),
  scanPriceImage(file).catch(...),
  recognizePriceText(file).catch(...),
])
```

That design matters because card identity and price OCR are independent. A photo may identify the card but fail price OCR, or price OCR may succeed while the card match needs manual selection. The UI merges what succeeds instead of making the whole scan fail.

5.5 Theme System

The theme type is:

```
type ThemeName = "graphite" | "wooper" | "paper" | "liquid";
```

The non-liquid themes are mostly CSS variable skins. The liquid theme is deeper: it changes the background, app surfaces, navigation, buttons, inputs, lock button, search portal, and app chrome. The global CSS contains a shared glass layer and then liquid-specific overrides under `[data-theme="liquid"]`.

The liquid theme uses `liquid-glass-react` through `LiquidGlassSurface`. This matters because a convincing glass effect needs more than a translucent background color. The current app wraps surfaces in a reusable glass component so that panels, nav buttons, and nested surfaces can share a consistent refraction model.

6 Frontend Styling and Liquid Theme

The style system is centralized in `src/app/globals.css`. It has:

- app-wide CSS variables for color, foreground, muted text, borders, inputs, panels, and accent colors;
- base utility classes such as panel, input shell, bottom nav, decision surface, and lock button;
- theme-specific blocks for graphite, wooper, paper, and liquid;
- special liquid selectors for glass panels, nested glass, bottom navigation, search result portal, theme options, inputs, buttons, and the lock button.

The liquid theme also introduces a background image variable:

```
--liquid-background-image  
--liquid-background-position  
--liquid-background-size
```

The goal is that the background exists once, behind the application, and the cells/panels refract it. The frontend then lets the user select a background from the liquid background options. This is important because glassmorphism only reads as glass if there is meaningful content behind the surface. A flat color behind a translucent panel just looks like opacity.

The theme work has also created a useful rule for future UI: if a surface is meant to look like glass, it should be wrapped by `LiquidGlassSurface` instead of duplicating border/background CSS manually. Manual overlays can produce the "two boxes" problem where one opaque cell is visible behind a glass layer.

7 Deal Calculation Model

The business logic is in `src/lib/calculations.ts`. This is the app's financial core.

7.1 Default Settings

The default settings include:

- yen divisor: 25
- selling fee: 5 percent

- buy threshold: 30 percent
- maybe threshold: 15 percent
- PSA grading fee: 220 DKK

The currency conversion formula is deliberately simple:

$$\text{costDkk} = \frac{\text{shopPriceYen}}{\text{yenDivisor}}$$

At the default divisor, 1,000 yen becomes 40 DKK.

7.2 Margin Formula

The calculator first applies the selling fee to the market value:

$$\text{netMarketValueDkk} = \text{marketValueDkk} \times (1 - \text{sellingFee})$$

Then it computes profit and margin:

$$\text{profitDkk} = \text{netMarketValueDkk} - \text{costDkk}$$

$$\text{marginPercent} = \begin{cases} 100 \times \frac{\text{profitDkk}}{\text{costDkk}}, & \text{if costDkk} > 0 \\ 0, & \text{otherwise} \end{cases}$$

7.3 Routes

The route model is important. The app does not only ask "is this card worth more than the shop price?" It asks which exit route is plausible:

In-hand condition	Candidate route	Extra cost
Raw	Raw flip	No grading fee
Raw	Grade to PSA 9	Add grading fee
Raw	Grade to PSA 10	Add grading fee
PSA 9 slab	PSA 9 slab resale	No grading fee
PSA 10 slab	PSA 10 slab resale	No grading fee

The route thresholds are not all the same. Raw grading routes require higher margin because they are riskier. Slab routes can tolerate lower margin because grade uncertainty is already resolved.

7.4 Warnings

The calculator emits warnings so that a single green decision cannot hide important risk:

Warning	Meaning
VALUES_MISSING	The app does not have enough market values to calculate a meaningful decision.
PSA_10_TRAP	Raw card only looks good if PSA 10 happens, while PSA 9/raw routes do not support the purchase.

<code>HIGH_END_RISK</code>	High-cost cards can be capped from BUY to MAYBE because the downside is larger.
<code>MANUAL_VALUES</code>	The user manually overrode values. The result is still valid but should be read as user-supplied.

These warnings are covered by tests in `src/lib/calculations.test.ts`.

8 Next.js Backend API Layer

The backend is implemented with App Router route handlers under `src/app/api`. The app uses route handlers instead of separate Express/FastAPI for the TypeScript side because the API endpoints are closely coupled to frontend types and app state.

Route	Role
<code>/api/card-search</code>	GET endpoint for manual card search. Calls <code>searchTcgDexCards</code> . Caches search promises for 10 minutes.
<code>/api/reference-prices</code>	POST endpoint. Accepts a card and optional filters, then calls <code>fetchReferencePricesServer</code> .
<code>/api/scan-card</code>	POST endpoint. Accepts an image file, validates it, and calls the local/card-provider scan pipeline. Uses Node runtime because it spawns Python.
<code>/api/price-ocr</code>	POST endpoint. Accepts an image file and proxies it to the local PaddleOCR GPU sidecar. Falls back to parsed text fields if needed.
<code>/api/cardmarket/raw-price</code>	Cardmarket raw price helper endpoint.
<code>/api/reverse-geocode</code>	Store/city helper endpoint.

The route handlers generally follow a thin-controller pattern: validate inputs, call a library function, return JSON. Most logic lives under `src/lib`, which keeps it testable.

9 Manual Card Search

Manual search is implemented in `src/lib/tcgdex.ts`. Despite the filename, it is not only TCGdex. It can combine:

- TCGdex card and set endpoints;
- PokemonTCG API search for English cards;
- TCGTracking set/product data for some marketplace-oriented mappings;
- hard-coded and dynamic set aliases.

The parser extracts:

- collector/local ID;
- name tokens;

- known set aliases;
- preferred set ID;
- language bucket;
- TCGTracking set IDs.

The score function rewards exact collector matches, known set matches, preferred language, name token matches, and source confidence. It also includes recency scoring for collector-only searches, which is useful when a shop label or OCR produces only a number.

This search layer is important as both a manual fallback and as a resolver for OCR-derived text candidates.

10 Reference Pricing System

10.1 Main Merge Function

Reference pricing is coordinated in `src/lib/reference-prices.server.ts`. The central function is `fetchReferencePricesServer`. It runs several sources in parallel:

- live Cardmarket pricing, if enabled;
- TCGdex pricing hashes;
- PokemonTCG API pricing fields;
- PokePrices;
- PriceCharting, especially for graded PSA values.

The first parallel pass asks Cardmarket live, TCGdex, PokemonTCG, and PokePrices. If PSA values are still missing, it then asks PriceCharting. This ordering keeps the app responsive while still giving graded references when available.

10.2 Currency Conversion

The code uses:

- EUR to DKK: 7.46
- USD to DKK: 7.00

These are simple fixed conversion constants, not a live FX feed. That is acceptable for a fast shopping tool but should be revisited if precision matters.

10.3 Source Priority

Raw prices are selected by source priority:

```
cardmarket -> tcgplayer -> pokeprices -> pricecharting
```

PSA values are selected mostly from:

```
pricecharting
pokeprices
```

depending on grade and source availability.

10.4 Cardmarket Live Pricing

Live Cardmarket pricing is in `src/lib/cardmarket-live.ts` plus the Python sidecar `tools/cardmarket_server/server.py`.

The TypeScript adapter builds a request containing:

- card name;
- collector number;
- Cardmarket product ID if known;
- language filter;
- minimum condition filter;
- search URL and query.

The Python sidecar logs into Cardmarket with Playwright, maintains a browser/cookie session, uses `curl_cffi` where possible, falls back to a browser page when needed, resolves the product page, applies language and condition URL filters, parses listings with BeautifulSoup, and returns the lowest matching listing.

This sidecar exists because direct marketplace API access is not assumed. It is powerful but operationally sensitive: it depends on credentials, session state, Cloudflare behavior, and HTML structure.

10.5 TCGdex Pricing

TCGdex details may include Cardmarket and TCGplayer aggregate prices. The app maps Cardmarket EUR values to raw DKK and TCGplayer USD values to raw DKK. When the user selected language or condition filters, the app prefers filtered Cardmarket listings if available and marks aggregate values as less exact.

There is a specific correction for some Japanese Scarlet/Violet secret ex cards where the non-holo aggregate can appear two decimals too low. The tests in `reference-prices.server.test.ts` cover this behavior.

10.6 PokemonTCG API Pricing

PokemonTCG is used mainly for English cards. It exposes TCGplayer and Cardmarket hashes for many cards. The app fetches card details by PokemonTCG ID when available and maps the best representative market value into DKK.

10.7 PokePrices

PokePrices is used for non-Asian cards. The code can query PokePrices' public Supabase RPC and scrape page metadata as a fallback. For some Japanese cards that correspond to English sets such as 151, Shiny Treasure/Paldean Fates, or Terastal/Prismatic, it contains helper logic to find English equivalents using TCGdex detail fields such as illustrator, HP, stage, type, and attack signature. However, the default policy avoids using English PokePrices pages for Asian-language cards unless a careful equivalent mapping is established.

10.8 PriceCharting

PriceCharting is the primary fallback for graded values. The app builds direct URLs when it can and search URLs when direct URLs are unsafe. It parses cells for raw, PSA 7, PSA 8, PSA 9, PSA 9.5, and PSA 10 values. Asian-language cards get special handling because PriceCharting distinguishes Japanese/Korean/Chinese pages.

11 Price OCR Scanner

There are two price OCR paths:

1. GPU server OCR via PaddleOCR through `/api/price-ocr`.
2. Browser fallback OCR via Tesseract in `src/lib/ocr.ts`.

11.1 Frontend Flow

When a deal photo is uploaded, the frontend calls `scanPriceImage(file)` and also runs `recognizePriceText(f` as a fallback. This lets the app recover if the local GPU price sidecar is down.

11.2 Next.js Price OCR Route

`src/app/api/price-ocr/route.ts` validates:

- the uploaded field exists;
- it is an image;
- it is under 10 MB.

Then it posts the file to:

```
PRICE_OCR_URL or http://127.0.0.1:8765/ocr-price
```

If the sidecar returns text and candidates, the route normalizes the response and chooses a yen price from:

- explicit `yenPrice` from the sidecar;
- numeric `prices`;
- parsed yen prices from returned text.

11.3 GPU Price OCR Sidecar

The sidecar is `tools/price_ocr_server/server.py`. It exposes:

- `/health`
- `/ocr-price`

The OCR engine is lazily initialized:

```
PaddleOCR(  
  device=os.getenv("POKEARB_PRICE_OCR_DEVICE", "gpu:0"),  
  lang=os.getenv("POKEARB_PRICE_OCR_LANG", "japan"),  
  text_detection_model_name="PP-OCRv5_server_det",  
  text_recognition_model_name="PP-OCRv5_server_rec",  
  use_doc_orientation_classify=False,  
  use_doc_unwarping=False,  
  use_textline_orientation=False,  
)
```

This is a deep learning OCR model stack. It has a text detector and text recognizer. It is not trained specifically on Pokemon cards, but it is strong enough to read common shop price tags.

11.4 White Price Tag Detection

Before OCR, the sidecar builds regions:

1. full image;
2. up to several white price-tag crops.

The crop detector looks for bright, low-saturation connected components with price-tag-like aspect ratios. It resizes large images down to a max dimension of 640 for detection, builds a binary mask of bright neutral pixels, finds connected components, filters by area/aspect/fill ratio, expands bounding boxes, and OCRs each crop.

This is a good pragmatic approach because Japanese shop stickers are often white or light colored and contain the price text in a concentrated region. OCR on the full photo can be distracted by card text and attacks; OCR on a likely sticker is much cleaner.

11.5 Yen Candidate Extraction

Both the Python sidecar and browser fallback use yen-specific extraction logic. They score candidates based on:

- labels such as tax-included, price, JPY, yen symbol, or yen word;
- whether the number has grouping separators;
- whether OCR confidence exists;
- whether the text came from a white-tag region;
- bounding box size;
- plausible yen range.

The parser handles common OCR confusions:

OCR text	Interpreted as	Reason
O or o	0	common OCR confusion in prices
I, l, or vertical bar	1	common OCR confusion in narrow digits
S or s	5	common OCR confusion
B	8	common OCR confusion

It also attempts to avoid card attack damage being merged into a price. For example, if OCR reads a price and a nearby attack damage value as one long digit run, the parser can split likely attack-damage suffixes such as 20, 80, 120, or 330.

12 Card Image Recognition System

12.1 Why Retrieval Instead of a Giant Classifier

The card identification problem looks like classification at first: choose one card from tens of thousands of prints. But a monolithic classifier would be the wrong default for this app.

Reasons:

- New cards release constantly. A classifier would need repeated retraining.
- Clean reference images already exist, so nearest-neighbor retrieval is natural.
- Real shop photos vary by sleeve, glare, angle, crop, lighting, and compression.
- Some cards differ only by set, collector number, language, holo variant, or small text details.
- The system needs top candidates and confidence evidence, not only one label.

The current design is therefore:

```
photo -> crop/normalize -> visual fingerprints + DINOv2 embeddings
      -> retrieve candidates -> optional OCR evidence -> rerank
      -> canonical print UID + marketplace queries + score breakdown
```

This means the model does not have to memorize every card. The gallery stores the card identities, and the model provides a strong visual similarity representation.

12.2 Data Sources

The card matcher can build galleries from TCGdex and from the official Japanese card search. The current broad Japanese coverage uses the official Japanese card search because TCGdex did not expose image-backed records for all Japanese cards.

The official Japanese gallery builder is `tools/card_matcher/card_matcher/official_jp.py`. It calls:

```
https://www.pokemon-card.com/card-search/resultAPI.php
```

with `regulation_detail=all`, fetches all result pages, reads card IDs, card names, thumbnail paths, derives a set-like ID from the image path, and downloads images concurrently.

The current official Japanese index has:

- 22,863 cards in `ja_official_image_index.json`;
- 22,863 fingerprints;
- 91,452 DINOv2 reference vectors;
- 4 embedding variants per card;
- 768 dimensions per embedding vector.

The embedding count is:

$$22,863 \times 4 = 91,452$$

The approximate uncompressed embedding memory is:

$$91,452 \times 768 \times 4 \approx 281 \text{ MB}$$

The compressed `.npz` file is around 260 MB.

12.3 Card Metadata

Card metadata is represented by `CardMetadata` in `tools/card_matcher/card_matcher/models.py`. It includes:

- internal ID;
- name;
- set ID;
- set name;
- local/collector ID;
- image URL;
- local image path;
- language;
- printed total;
- variant class;
- rarity;
- regulation mark;
- TCGdex ID;
- PokemonTCG API ID;
- marketplace URL.

The canonical print UID is:

```
<language>:<set_id>:<collector_number_or_localId>:<variant_class>
```

For example:

```
ja:m4:50085:standard
```

This UID is an internal stable identifier. Marketplace product IDs are useful when available, but they are not the core recognition identity.

12.4 Current Japanese Coverage Caveat

The official Japanese gallery gives broad visual coverage, but it currently stores the official card search ID as `local_id`. That is not always the printed collector number visible on the card. This is acceptable for visual matching and internal identity, but it is not perfect for marketplace search or user explanation.

The next metadata improvement should parse or reconcile:

- printed collector number;
- printed set total;
- exact set name;
- rarity;
- variant/foil type;
- product-page marketplace identity where available.

13 Image Preprocessing and Geometry

Card photo normalization is in `tools/card_matcher/card_matcher/image_ops.py`.

13.1 Target Size

All normalized cards are resized to:

```
420 x 586
```

That target has the expected Pokemon card aspect ratio and gives a stable input shape for hashes, histograms, ORB, and embeddings.

13.2 Candidate Crops

For a query photo, the matcher creates several normalized variants:

1. contour-based perspective crops;
2. a center card crop;
3. the full image oriented portrait.

This is important for real phone photos. If contour detection fails because of glare or a cluttered background, the center/full variants can still sometimes match. If contour detection succeeds, OCR and embeddings get a cleaner card image.

13.3 Contour Detection

The contour pipeline uses:

- grayscale conversion;
- CLAHE equalization;
- Gaussian blur;
- Canny edge detection at multiple thresholds;
- adaptive thresholding;
- dilation;
- contour extraction;
- minimum/maximum area filtering;
- aspect ratio filtering;
- `cv2.minAreaRect`;
- four-point perspective transform.

The perspective transform uses OpenCV's `getPerspectiveTransform` and `warpPerspective`. This takes a quadrilateral card region and turns it into a front-facing rectangle.

13.4 Why No Trained Detector Yet

The current matcher uses OpenCV geometry rather than a trained YOLO/RT-DETR detector. This was a reasonable MVP choice because it required no labeled dataset. However, for binder pages, extreme angles, partial occlusion, and busy shop backgrounds, a trained single-class detector or oriented detector will be more robust.

The codebase already has a placeholder/direction for detector work in the card matcher package, but the production path today is OpenCV crop normalization.

14 Classical Visual Features

Before DINOv2 embeddings, the matcher computes classical image similarity signals. These are still useful because they are cheap and provide complementary evidence.

14.1 Perceptual Hashes

The matcher computes:

- pHash;
- dHash.

The distance is a Hamming distance between hash bit patterns. The code converts the distance into a score:

$$\text{hashScore} = \max(0, \min(1, 1 - \frac{\text{distance}}{64}))$$

Hashes are fast and good for near-identical clean images, but they are brittle under perspective, glare, and crop changes. That is why they are only one part of the score.

14.2 Color Histogram

The matcher computes an HSV histogram with:

```
H bins = 16
S bins = 8
V bins = 8
```

OpenCV histogram correlation is normalized into a 0 to 1 similarity. Histograms are useful for broad visual similarity: a green card, dark full-art card, or bright trainer card often has a recognizable color distribution. They are not sufficient for identity because many cards share similar palettes.

14.3 ORB Keypoint Matching

The rerank stage loads the candidate reference image, normalizes it, computes ORB keypoints/descriptors for query and reference, and compares descriptors with a brute-force Hamming matcher.

ORB helps when there are distinctive art or text features. It is still affected by blur and glare, but it is often more identity-specific than a color histogram.

15 DINOv2 Embeddings

15.1 What the Embedding Model Does

The current deep visual model is:

```
facebook/dinov2-base
```

It is loaded through Hugging Face Transformers in `tools/card_matcher/card_matcher/embeddings.py`. It is used as a feature extractor, not as a trained Pokemon classifier.

For each image, the model outputs a dense vector. The code uses:

- `outputs.image_embeds` if available;
- else `outputs.pooler_output`;
- else the CLS token from `last_hidden_state`.

The vector is L2-normalized:

$$\hat{x} = \frac{x}{\|x\|_2}$$

Similarity between a query and reference is cosine similarity:

$$\text{cosine}(q, r) = \hat{q} \cdot \hat{r}$$

The matcher maps cosine from roughly $[-1, 1]$ into $[0, 1]$:

$$\text{embeddingScore} = \max(0, \min(1, \frac{\text{cosine} + 1}{2}))$$

15.2 Why DINOv2 Helps

DINOv2 is useful here because it produces robust visual features without being trained on the exact card IDs. It can recognize that two images share the same art/layout even if one is a clean reference scan and the other is a phone photo with perspective, glare, and background clutter.

In this app, DINOv2 is used to answer:

"Which reference images look visually closest to this phone crop?"

It does not know card names, prices, or collector numbers by itself. Those come from the metadata attached to the nearest reference image.

15.3 Reference Embedding Variants

Each reference card gets four embeddings:

1. full normalized card;
2. 3.5 percent margin crop resized back to card size;
3. 7 percent margin crop resized back to card size;
4. artwork region crop resized back to card size.

The artwork crop covers approximately:

```
top:    12 percent
bottom: 58 percent
left:   8 percent
right:  92 percent
```

This is a practical trick. Camera photos may crop borders, sleeves may obscure edges, and text may be blurred. The art region is often the most distinctive part of the card.

15.4 Query Embedding Variants

The query image is also represented by multiple candidates: contour crops, center crop, and full image. The model embeds these variants, compares each against the reference matrix, and keeps the maximum score per card.

This means a bad crop does not automatically ruin the query as long as another crop variant is usable.

15.5 Current Search Implementation

The embedding index is stored as a NumPy `.npz` file:

- `card_ids`: one card ID per vector;
- `embeddings`: float32 matrix;
- `model_name`: model identifier.

At match time, query embeddings are computed on CUDA if configured, then cosine similarities are computed with NumPy:

```
reference @ query_vector
```

This is exact brute-force retrieval over the current matrix. It is simple and accurate. For larger indexes or lower latency, `faiss_store.py` and `build_faiss.py` provide the direction for FAISS-backed search.

15.6 RTX 5090 Role

The RTX 5090 is valuable in two places:

- building the reference embedding index for tens of thousands of cards;
- embedding query photos quickly at runtime.

The current runtime does not fine-tune DINOv2. It loads a pretrained model and runs inference. The heavy training path would only become necessary if the retrieval quality plateaus and real-photo labels show that fine-tuning or a learned metric model is needed.

16 Matcher Scoring and Reranking

16.1 Coarse Score

For each query crop and reference fingerprint, the matcher computes:

$$\text{coarseScore} = 0.72 \times \text{hashScore} + 0.28 \times \text{histogramScore}$$

If embeddings are available, it blends in DINOv2:

$$\text{coarseScore}' = (1 - w_e) \times \text{coarseScore} + w_e \times \text{embeddingScore}$$

The default embedding weight is:

$$w_e = 0.52$$

The code clamps that weight to at most 0.85 so embeddings cannot completely erase the classical signals.

16.2 Rerank Stage

The matcher keeps the best coarse candidates, loads their reference images, computes ORB similarity, and builds:

$$\text{baseScore} = 0.62 \times \text{coarseScore} + 0.38 \times \text{orbScore}$$

If embeddings are available:

$$\text{finalVisualScore} = (1 - w_e) \times \text{baseScore} + w_e \times \text{embeddingScore}$$

This visual score becomes the input to the V2 contract reranker.

16.3 V2 Reranker

The V2 reranker is in `tools/card_matcher/card_matcher/reranker.py`. It scores:

- visual similarity;
- OCR name similarity;
- collector number similarity;
- set-symbol consistency;
- language consistency;
- variant consistency.

The default weights are:

$$\begin{aligned} w_v &= 0.45 \\ w_n &= 0.15 \\ w_{num} &= 0.20 \\ w_s &= 0.10 \\ w_l &= 0.05 \\ w_{var} &= 0.05 \end{aligned}$$

If no OCR fields are available, the reranker switches to:

$$w_v = 1.0$$

and all text/variant weights become zero. This is exactly what the hosted local image matcher usually does today because card ROI OCR is optional and not passed by default from `scan-card.server.ts`.

If collector-number OCR is enabled and confidence is high, the reranker increases the number weight because collector number is a strong print identity signal.

16.4 Unimplemented or Placeholder Evidence

Two feature channels are currently placeholders:

- set-symbol score;
- variant/foil score.

They are part of the contract and weighting model but currently return zero unless future classifiers populate them. This is good architecture because it keeps the output shape stable while leaving room for the next model additions.

17 Card OCR and ROI Extraction

17.1 Purpose

Card OCR is different from price OCR. Price OCR tries to read shop stickers. Card OCR tries to read card fields:

- name;
- collector number;
- set-symbol neighborhood;
- attack text;
- language/script;
- variant/foil surface regions.

17.2 Current Implementation

The card matcher has ROI helpers in:

- `card_matcher/roi.py`
- `card_matcher/ocr.py`

When `match_card_v2.py` is run with `--ocr`, it normalizes the best crop, sends ROIs through PaddleOCR, and emits `OcrFeatures`. These features can then influence reranking.

17.3 Current Runtime Limitation

The Next.js route currently calls `match_card_v2.py` with `image/index/top/json` and optional `embedding/debug` arguments. It does not add `--ocr` by default. This means the hosted path is primarily visual retrieval today. Price OCR is fully integrated; card-field OCR is available in the Python CLI but not yet always active from the web app.

This is a sensible staged rollout because visual retrieval is faster and less brittle. Once OCR is validated on a real-photo set, the hosted route can enable it or make it conditional.

18 Recognition Output Contract

The V2 matcher outputs a structured JSON contract from `contract.py`. The key fields are:

```
{
  "canonical_print_uid": "...",
  "match_confidence": 0.99,
  "needs_review": false,
  "language": "ja",
  "recognized_fields": {
    "name": "...",
    "collector_number": "...",
    "set_id": "...",
    "set_name": "...",
    "variant_class": "standard",
    "language": "ja"
  },
  "source_ids": {
    "pokemon_tcg_api_id": null,
    "tcgdex_id": null,
    "tcgplayer_product_id": null,
    "tcgplayer_group_id": null,
    "cardmarket_product_ref": null
  },
  "marketplaces": {
    "north_america": {
      "product_id": null,
      "group_id": null,
      "query": "...",
      "resolution_mode": "query"
    },
    "europe": {
      "product_ref": null,
      "query": "...",
      "resolution_mode": "query"
    }
  },
  "evidence": {
    "visual_similarity": 0.99,
    "name_score": 0,
    "number_score": 0,
    "set_symbol_score": 0,
    "language_score": 0,
    "variant_score": 0
  },
  "candidates": [...]
}
```

This contract is much better than returning one opaque string. It preserves:

- the chosen print identity;
- confidence;
- why it matched;
- backup candidates;
- marketplace search strings;
- whether manual review is needed.

19 Next.js Integration for Card Scanning

19.1 Route

`/api/scan-card` is implemented in `src/app/api/scan-card/route.ts`. It validates the file and delegates to `scanCardImageServer`.

It sets:

```
export const dynamic = "force-dynamic";
export const runtime = "nodejs";
```

Node runtime is necessary because the route spawns the Python matcher with `child_process.execFile`.

19.2 Provider Order

The default provider order in `scan-card.server.ts` is:

```
local-image
gibltcg
```

Google OCR can also be used if configured, but it is not in the default order unless the provider list changes. GiblTCG requires an API key. The local image matcher is the main path.

19.3 Local Matcher Execution

The TypeScript backend writes the uploaded image to a temporary directory, builds command-line arguments, and runs:

```
python match_card_v2.py <temp-image> --index <index> --top 8 --json
```

If an embedding index exists, it also adds:

```
--embedding-index <embedding-index> --device cuda
```

If debug is configured, it adds:

```
--debug-dir <directory>
```

The temporary directory is removed after execution.

19.4 Parsing and Mapping

The matcher output can be either the older V1 array shape or the V2 JSON object shape. The parser supports both. For V2 it maps each candidate into the frontend's `CardScanCandidate` shape, preserving:

- card identity;
- confidence;
- canonical print UID;
- recognition evidence;
- marketplace query contract.

20 Scan Result Merging

When multiple providers produce card candidates, `src/lib/scan-merge.ts` merges them. The logic deduplicates by card ID when a candidate has a resolved card, or by query/source when it only has text.

The app only auto-selects candidates that:

- are not local OCR-only;
- have a resolved card;
- meet the confidence threshold, default 0.8.

This protects the user from weak OCR-only guesses being selected without review.

21 Marketplace Query Strategy

The recognition system emits marketplace-ready queries rather than assuming marketplace IDs always exist.

21.1 North America

The north America query template is:

```
<name> <set_name> <collector_number> <variant>
```

If TCGplayer product IDs or group IDs become available later, they can be stored separately. They are not required for the recognition pipeline.

21.2 Europe

The Europe/Cardmarket query template is:

```
<name> <collector_number> <set_name>
```

If a curated Cardmarket short code exists, the query can become:

```
<name> <short_code><collector_number>
```

This distinction matters because marketplace abbreviations are not always the same as TCGdex set IDs or PokemonTCG online codes.

22 History and Persistence

The app stores saved deals in `localStorage`. A saved deal contains the calculated decision, route, card info, values, price, store context, and notes. This keeps the app simple and offline-friendly from the user's perspective.

There is no backend database for deals yet. That means:

- history is device/browser-local;
- clearing browser storage clears history;
- there is no automatic sync between desktop and phone;
- export/import would be a useful future feature.

23 Testing Coverage

The TypeScript side has a focused Vitest suite:

Test file	Coverage
<code>calculations.test.ts</code>	Deal formulas, grading fees, warnings, thresholds, input cleanup, settings migration.
<code>ocr.test.ts</code>	Collector extraction, set code extraction, yen parsing, attack-damage false positives, PSA cert context.
<code>tcgdex.test.ts</code>	Search parsing, set aliases, language buckets, scoring behavior.
<code>reference-prices.server.test.ts</code>	Price merging, Cardmarket aggregate/live behavior, Japanese pricing correction.
<code>cardmarket-live.test.ts</code>	Sidecar request construction and live response mapping.
<code>cardmarket-url.test.ts</code>	Cardmarket search URL and filter generation.
<code>pricecharting.test.ts</code>	PriceCharting URL generation and HTML parsing.
<code>pokeprices.test.ts</code>	PokePrices slug/search/helpers and Asian-card policy.
<code>market-price.test.ts</code>	Market price field selection.
<code>scan-merge.test.ts</code>	Candidate merge and auto-select behavior.

The Python card matcher also has tests for identity, catalog, official Japanese source, reranker, and contract modules under `tools/card_matcher/tests`.

24 Current Verification State

The app has been smoke-tested through `/api/scan-card` against the official Japanese index. A known official image returned the expected official Japanese candidate, a V2 canonical UID, high confidence, candidate list, and recognition contract.

That verifies:

- Next.js upload route works;
- Python matcher can be spawned from Node;
- official Japanese image index is readable;
- DINOv2 embedding index is readable;
- CUDA embedding inference works in the configured environment;
- V2 JSON output maps into frontend scan candidates.

It does not prove real shop-photo robustness by itself. That requires a labeled photo benchmark containing camera shots, sleeves, glare, binder pages, partial crops, and hard negatives.

25 Deep Learning Inventory

Model/system		Current role	Notes
DINOv2 base		Visual embedding retrieval	Main deep visual model. Used as frozen feature extractor. No Pokemon-specific training yet. Produces 768-dimensional normalized embeddings.
PaddleOCR OCR	price	Price-tag OCR	Runs in <code>tools/price_ocr_server</code> . Uses PP-OCRv5 server detection and recognition models on GPU.
PaddleOCR card ROI OCR	card	Optional card-field OCR	Available in <code>match_card_v2.py --ocr</code> . Not always enabled by the web route yet.
Tesseract.js		Browser fallback OCR	Runs client-side for fallback text/price extraction. Less accurate on real photos but useful when local sidecar is unavailable.
Future DETR	YOLO/RT-	Detector candidate	Not trained/integrated yet. Would improve card localization on binder pages and cluttered photos.
Future set-symbol classifier		Reranker evidence	Contract weight exists; classifier not implemented yet.
Future classifier	holo/variant	Variant resolver	Contract weight exists; classifier not implemented yet.

26 What Has Not Been Trained From Scratch

No current system has been trained from scratch on a custom Pokemon dataset. The app uses:

- pretrained DINOv2 for embeddings;
- pretrained PaddleOCR for OCR;
- classical OpenCV for geometry and local visual signals;
- metadata/reference-image retrieval for identity.

This is the right first approach. Training from scratch is expensive and unnecessary until a labeled benchmark proves a specific failure mode that pretrained retrieval cannot solve.

27 Where Training May Become Useful

Training or fine-tuning becomes justified if benchmark data shows one of these problems:

- OpenCV crop detection frequently misses cards in real phone photos.
- Same-art or same-name reprints are visually confused and OCR is not enough.
- Foil/reverse/holo variants need reliable separation from shop photos.
- Japanese collector-number OCR is weak under glare/sleeves.
- Binder/page mode needs multiple card detection.

The likely training targets are:

1. single-class card detector, probably YOLO11 or similar, trained on real photos with boxes/quads;
2. set-symbol classifier trained from set symbol assets plus real crops;
3. finish/variant classifier trained from labeled real photos;
4. lightweight learned reranker trained over visual, OCR, set, language, and variant feature scores;
5. optional metric fine-tuning if DINOv2 retrieval is not good enough for hard negatives.

28 Failure Modes

28.1 Recognition Failure Modes

Current expected failures include:

- glare covering the artwork or collector number;
- cards inside highly reflective sleeves/toploaders;
- photo too close or cropped to only part of the card;
- multiple cards in one image when the user expects one;
- same artwork across languages/sets;
- very similar full-art variants;
- official gallery metadata not exposing printed collector numbers;
- cards missing from the downloaded reference gallery.

28.2 Price OCR Failure Modes

Expected price OCR failures include:

- handwritten or stylized price tags;
- price sticker over a busy holo region;
- price tag color not detected by the white-tag crop heuristic;
- attack damage or card numbers mistaken for price;
- several prices visible at once;
- discount labels where the crossed-out original price is more visible than the sale price.

28.3 Pricing Failure Modes

Market pricing is inherently noisy. Failures include:

- stale aggregate prices;
- wrong language page;
- wrong variant page;
- thin sales history;
- low sample sizes for PSA grades;
- Cardmarket login/session/Cloudflare issues;
- marketplace HTML layout changes.

29 Recommended Evaluation Harness

The next serious quality step is not more guessing. It is a real-photo labeled benchmark.

29.1 Dataset

Create a dataset with:

- single-card clean phone photos;
- angled desk shots;
- shop shelf photos;
- sleeved cards;
- top-loader cards;
- binder pages;
- Japanese cards across eras;
- English cards;
- cards with visible price tags;
- cards without visible price tags;
- non-Pokemon negatives.

29.2 Labels

Each labeled sample should include:

- image path;
- expected canonical UID or official ID;
- printed collector number if known;
- language;
- expected yen price if visible;
- notes: glare, sleeve, binder, crop, low light, etc.

29.3 Metrics

Track:

- card top-1 accuracy;
- card top-5 accuracy;
- card top-20 accuracy;
- mean reciprocal rank;
- price OCR exact accuracy;
- price OCR within-one-candidate accuracy;
- crop success rate;
- median and p95 latency;
- failure reason categories.

The Python `benchmark.py` already reports top-1, top-5, top-20, and MRR for card matching. It should be expanded to also run price OCR and produce scenario-sliced reports.

30 Security and Operational Considerations

30.1 Local-Only Assumption

The app currently assumes a local trusted environment. It spawns Python scripts and can access local caches. That is acceptable for a personal local tool, but it would need hardening before becoming a public service.

30.2 Upload Limits

The scan routes limit image uploads to 10 MB. This helps avoid accidental huge uploads and keeps sidecar processing bounded.

30.3 External Credentials

Credentials may be needed for:

- GIBITCG API;
- Google Vision API;
- Cardmarket sidecar login;
- optional PokePrices Supabase anon key override.

These should remain in local environment files, not in source code.

30.4 Marketplace Scraping

The Cardmarket sidecar automates a logged-in browser and parses HTML. This is fragile and should be treated as a local helper, not a guaranteed API contract. The app already degrades to aggregate pricing/search URLs when live pricing is unavailable.

31 Recommended Roadmap

31.1 Near-Term

1. Add a labeled real-photo benchmark with at least 200 to 500 images.
2. Enable optional card ROI OCR in a controlled benchmark mode and measure whether it improves top-1/top-5 accuracy.
3. Improve official Japanese metadata by scraping/reconciling printed collector number and set total.
4. Add a visible recognition details/debug panel in development mode showing top candidates and evidence.
5. Add price OCR benchmark cases from real shop photos.

31.2 Medium-Term

1. Train a single-class card detector for real photos and binder pages.
2. Add set-symbol classifier.
3. Add foil/variant classifier.
4. Move vector search to FAISS or Qdrant once latency or filtering requires it.
5. Add export/import for saved deals and settings.

31.3 Long-Term

1. Learned reranker trained from real scan outcomes.
2. Fine-tuned visual embedding model if DINOv2 hard-negative performance is insufficient.
3. Full English and Japanese gallery with reconciled marketplace mappings.
4. Offline/on-device mobile mode if needed.
5. Multi-card binder scan workflow with batch results.

32 Glossary

Term	Meaning
Canonical print UID	Internal identity string combining language, set ID, collector/local ID, and variant class.
Reference gallery	Local collection of clean card images and metadata used for retrieval.
DINOv2 embedding	Dense vector representation of an image produced by pre-trained DINOv2.
Cosine similarity	Dot product of normalized vectors; used to compare query and reference embeddings.
ORB	Classical local feature detector/descriptor used for image matching.

pHash/dHash	Perceptual image hashes used for cheap approximate similarity.
ROI	Region of interest, such as the name box or collector-number area.
Reranker	Logic that combines visual, OCR, language, set, and variant signals into a final score.
Sidocar	Separate local Python service called by the Next.js app.

33 Key Files

File	Why it matters
<code>src/app/page.tsx</code>	Main frontend application.
<code>src/app/globals.css</code>	Theme and liquid glass styling.
<code>src/lib/calculations.ts</code>	Deal and route calculation.
<code>src/lib/ocr.ts</code>	Browser OCR and yen/card text extraction helpers.
<code>src/lib/price-ocr.ts</code>	Client wrapper for GPU price OCR route.
<code>src/lib/scan-card.server.ts</code>	Backend card scan orchestration and Python matcher execution.
<code>src/lib/reference-prices.server.ts</code>	Pricing source orchestration and merge logic.
<code>src/lib/tcgdex.ts</code>	Manual card search and query parsing.
<code>tools/card_matcher/match_card_v2.py</code>	V2 card recognition CLI and JSON contract entry point.
<code>tools/card_matcher/card_matcher/image_ops.py</code>	Card crop normalization, hashing, histograms, ORB.
<code>tools/card_matcher/card_matcher/embeddings.py</code>	DINOv2 embedding model and embedding index.
<code>tools/card_matcher/card_matcher/matcher.py</code>	Visual matching pipeline.
<code>tools/card_matcher/card_matcher/reranker.py</code>	V2 feature reranking.
<code>tools/card_matcher/card_matcher/contract.py</code>	Recognition JSON output contract.
<code>tools/card_matcher/card_matcher/official_jp.py</code>	Official Japanese gallery ingestion.
<code>tools/price_ocr_server/server.py</code>	PaddleOCR price OCR sidocar.
<code>tools/cardmarket_server/server.py</code>	Cardmarket live listing sidocar.

34 Conclusion

PokeArb Japan is already structured around the right architecture for the problem. The frontend is a focused mobile deal tool. The backend separates search, pricing, card scanning, and OCR into testable modules. The card recognition stack uses the maintainable retrieval approach: reference gallery plus visual embeddings plus reranking, instead of a brittle giant classifier. The

price OCR stack uses a practical hybrid of GPU OCR, tag crop detection, and yen-specific parsing.

The most important next step is not to train a huge model immediately. It is to build a real-photo benchmark and measure the current system. Once failures are quantified, the app can add the right specialized models: detector, set-symbol classifier, variant classifier, or learned reranker. With the RTX 5090 and the existing MNE environment, the hardware is not the bottleneck. The bottleneck is labeled evaluation data and precise metadata reconciliation, especially for Japanese cards.

A Appendix: Useful Commands

A.1 Run App on Port 3000

```
npm run dev -- --hostname 0.0.0.0 --port 3000
```

A.2 Build Official Japanese Gallery

```
conda run -n MNE python tools/card_matcher/build_official_jp_gallery.py
```

A.3 Build DINOv2 Embeddings

```
conda run -n MNE python tools/card_matcher/build_embeddings.py ^  
  --index tools/card_matcher/cache/ja_official_image_index.json ^  
  --language ja_official ^  
  --device cuda ^  
  --batch-size 128 ^  
  --clean-scans
```

A.4 Match One Photo

```
conda run -n MNE python tools/card_matcher/match_card_v2.py path\to\photo.jpg ^  
  --index tools/card_matcher/cache/ja_official_image_index.json ^  
  --embedding-index tools/card_matcher/cache/ja_official_embeddings_facebook_dinov2-  
    base.npz ^  
  --device cuda ^  
  --top 8 ^  
  --json
```

A.5 Run TypeScript Tests

```
npm test
```

A.6 Run Card Matcher Benchmark

```
conda run -n MNE python tools/card_matcher/benchmark.py samples\labels.csv --top 20
```